DataCenter Impact Map

Cartographie interactive de l'empreinte environnementale des datacenters mondiaux

🏆 **Hack for Good** Hackathon étudiant	🤝 **Défi ADEME** Open Data University S4

📋 Sommaire

1. Contexte et problématique 3
2. Vision du produit 4
3. Données disponibles 5
4. Architecture technique 6
5. Plan de réalisation 8
6. Structure des fichiers 10
7. Code de démarrage 11
8. Présentation jury (5 min) 14
9. Points de vigilance 15

1. Contexte et problématique

1.1 Le problème

L'intelligence artificielle générative repose sur une infrastructure physique massive : des milliers de datacenters dispersés mondialement, dont l'impact environnemental reste largement invisible pour le grand public.

Chiffres clés 2024

- 415 TWh consommés par les datacenters mondiaux (AIE 2024)
- 1,5% de la consommation électrique mondiale
- +12% de croissance annuelle
- Doublement prévu d'ici 2030 (→ 945 TWh)
- Google : +48% d'émissions entre 2019 et 2024
- Microsoft : +29% entre 2020 et 2023

Impacts méconnus

- Consommation d'eau massive pour le refroidissement
- Occupation foncière dans des zones parfois en tension
- Extraction de minerais rares pour les équipements
- Renouvellement prématuré des terminaux
- Opacité totale des grands acteurs sur leurs données réelles

1.2 Alignement avec le défi ADEME

Le défi proposé par l'ADEME pose la question : « Comment mobiliser les données existantes pour éclairer le débat public sur l'impact environnemental de l'IA ? »

Notre réponse
→ Rendre visible ce qui est invisible : localiser, quantifier et contextualiser l'impact des datacenters
→ Confronter les discours des entreprises à leurs impacts réels (Google, Microsoft, AWS...)
→ Offrir un outil pédagogique accessible à tout citoyen, sans expertise technique
→ S'appuyer exclusivement sur des données ouvertes et vérifiables

2. Vision du produit

2.1 Ce que l'utilisateur voit

Une carte mondiale interactive où chaque point représente un datacenter. En un coup d'œil, l'utilisateur comprend :

- Où sont concentrés les datacenters des grands acteurs de l'IA
- Quel est leur impact carbone selon le mix énergétique local
- Lesquels sont implantés dans des zones de stress hydrique
- L'écart entre les engagements publics et les données réelles

2.2 Fonctionnalités cibles

Fonctionnalité	Priorité	Description
Carte interactive mondiale	🔴 MVP	Points colorés par intensité carbone, clustering par zoom
Popup détaillée par datacenter	🔴 MVP	Opérateur, puissance, PUE, CO_2 estimé, source eau
Filtre par opérateur	🔴 MVP	Google / AWS / Azure / Meta / OVH / autres
Code couleur carbone	🔴 MVP	Vert (renouvelable) → Rouge (charbon/gaz)
Indicateur stress hydrique	🟡 Important	Overlay WRI Aqueduct sur les zones de tension
Panneau Discours vs Réalité	🟡 Important	Engagement CO_2 annoncé vs émissions mesurées
Statistiques globales	🟡 Important	Total TWh, CO_2, eau — chiffres en temps réel
Comparaison régionale	🟢 Bonus	Europe vs US vs Asie — mix énergétique et impact
Timeline de croissance	🟢 Bonus	Évolution du nombre de datacenters par année

3. Données disponibles

3.1 Sources principales

Source	Ce qu'elle fournit	Format	URL
OpenStreetMap (Overpass API)	Localisation de +10 000 datacenters mondiaux	GeoJSON / JSON	overpass-api.de
Electricity Maps	Intensité carbone par pays/région en temps réel	API REST (gratuit limité)	electricitymaps.com
WRI Aqueduct	Stress hydrique par bassin versant mondial	GeoJSON	wri.org/aqueduct
ODRE (data.gouv.fr)	Consommation électrique datacenters France	CSV / API	odre.opendatasoft.com
Google ESG Reports	PUE, consommation eau, émissions par région	PDF / Web	sustainability.google
Microsoft ESG Reports	Idem Microsoft Azure	PDF / Web	microsoft.com/esg
Boavizta	Modèles de calcul d'empreinte open source	Python lib / API	boavizta.org
Opcore	Données réelles de 3 DCs (conso + refroidissement)	CSV	opcore.com
IEA Energy and AI 2025	Stats mondiales datacenters et IA	PDF	iea.org

3.2 Modèle de données — structure GeoJSON

Chaque datacenter est représenté par un Feature GeoJSON enrichi :

```
{
  "type": "Feature",
  "geometry": { "type": "Point", "coordinates": [-95.86, 41.26] },
  "properties": {
    "name": "Google Council Bluffs",
    "operator": "Google",
    "country": "US",
    "region": "Iowa",
    "capacity_mw": 290,
    "pue": 1.12,
    "carbon_intensity_gco2_kwh": 412,
    "co2_annual_tonnes": 268000,
    "water_withdrawal_m3_year": 1800000,
    "water_stress_level": "low",
    "energy_source_pct_renewable": 61,
    "operator_commitment_net_zero": "2030",
    "source": "Google ESG 2024 + OSM",
    "last_updated": "2024-01"
  }
```

```
}
```

4. Architecture technique

4.1 Stack choisie

Frontend (carte)

- 🗺️ Mapbox GL JS v3
- 💻 HTML / CSS / Vanilla JS
- ✅ Pas de framework — moins de complexité
- 🚀 GitHub Pages pour le déploiement

Données

- 📦 GeoJSON statique enrichi
- ⚡ API Electricity Maps (temps réel carbone)
- 🐍 Script Python de préparation des données

Pourquoi Mapbox GL JS ?

- Contrôle total sur le rendu (vs Kepler.gl trop opaque)
- Documentation excellente, exemples abondants
- Clustering intégré (supercluster)
- Popups, filtres, couches — tout natif
- Gratuit jusqu'à 50 000 map loads/mois
- Déployable en fichiers statiques purs

4.2 Structure du projet

```
datacenter-map/
├── index.html              ← page principale
├── style.css               ← styles globaux
├── js/
│   ├── main.js             ← init carte Mapbox
│   ├── layers.js           ← couches (points, clusters, overlay)
│   ├── popup.js            ← contenu des popups
│   ├── filters.js          ← logique de filtrage
│   ├── stats.js            ← panneau statistiques
│   └── api.js              ← appels Electricity Maps
├── data/
│   ├── datacenters.geojson ← données enrichies (généré par Python)
│   └── water_stress.geojson← overlay stress hydrique WRI
├── scripts/
│   ├── fetch_osm.py        ← extraction OpenStreetMap
│   ├── enrich_data.py      ← enrichissement (PUE, carbone, eau)
│   └── merge.py            ← fusion des sources
└── README.md
```

4.3 Flux de données

🌐 **OSM / Rapports ESG** → 🐍 **Scripts Python (enrichissement)** → 📦 **datacenters.geojson** → 🗺️ **Mapbox GL JS**

5. Plan de réalisation

Le projet est structuré en 4 phases pouvant être menées en parallèle par 2 sous-équipes (Data / Frontend).

Phase 1

Préparation des données (Python) — ~3h

→ Extraction OSM via Overpass API → ~10 000 datacenters bruts
→ Nettoyage et déduplication (pandas)
→ Enrichissement manuel des 100 plus grands (Google, AWS, Azure, Meta, OVH...)
→ Calcul CO_2 estimé = capacity_mw × PUE × hours × carbon_intensity
→ Croisement avec WRI Aqueduct pour le stress hydrique
→ Export final : datacenters.geojson

Phase 2

Carte de base Mapbox — ~2h

→ Init projet HTML/CSS/JS + token Mapbox
→ Chargement du GeoJSON et affichage des points
→ Code couleur par intensité carbone (gradient vert → rouge)
→ Clustering automatique au dézoom
→ Popup au clic avec données brutes

Phase 3

Fonctionnalités avancées — ~3h

→ Filtres par opérateur (barre latérale)
→ Overlay stress hydrique (couche polygon WRI)
→ Panneau stats globales (total TWh, CO_2, eau)
→ Panneau Discours vs Réalité (table comparaison engagements/réel)
→ Intégration API Electricity Maps (carbone temps réel)

Phase 4

Finitions et démo — ~2h

→ Design UI/UX (dark mode, typographie, icônes)
→ Déploiement GitHub Pages
→ Préparation de la démo jury (scénario 5 min)
→ Vérification des sources citées
→ README et documentation data

⚠️ Règle d'or pour le hackathon

Le GeoJSON statique est la priorité absolue. Sans données, pas de carte. L'équipe data doit avoir un fichier exploitable dans les 2 premières heures, même avec seulement 50 datacenters enrichis.

Mieux vaut 50 datacenters parfaitement documentés que 10 000 points sans informations.

6. Structure des fichiers détaillée

6.1 index.html — squelette

Structure minimale de la page principale :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>DataCenter Impact Map</title>
  <link href="https://api.mapbox.com/mapbox-gl-js/v3.x/mapbox-gl.css"
rel="stylesheet">
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div id="sidebar">          <!-- filtres + stats --></div>
  <div id="map"></div>         <!-- carte plein écran -->
  <div id="popup-panel"></div> <!-- popup datacenter -->
  <script
src="https://api.mapbox.com/mapbox-gl-js/v3.x/mapbox-gl.js"></script>
  <script type="module" src="js/main.js"></script>
</body>
</html>
```

6.2 Responsabilités des fichiers JS

Fichier	Responsabilité
`main.js`	Initialisation Mapbox, chargement du GeoJSON, orchestration des modules
`layers.js`	Ajout des couches (cercles colorés, clusters, overlay hydrique), gestion des expressions de style
`popup.js`	Construction du HTML de la popup au clic sur un point (données + mise en forme)
`filters.js`	Logique de filtrage par opérateur, pays, intensité carbone
`stats.js`	Calcul et affichage des totaux globaux (TWh, CO_2, eau) en temps réel selon filtres actifs
`api.js`	Appels à l'API Electricity Maps pour récupérer l'intensité carbone en temps réel

7. Code de démarrage

7.1 Script Python — extraction OSM (fetch_osm.py)

```
import requests, json

OVERPASS_URL = "https://overpass-api.de/api/interpreter"

query = """
[out:json][timeout:60];
(
  node["facility"="data_center"];
  node["building"="data_center"];
  way["facility"="data_center"];
);
out center tags;
"""

resp = requests.post(OVERPASS_URL, data={"data": query})
elements = resp.json()["elements"]

features = []
for el in elements:
    lat = el.get("lat") or el.get("center", {}).get("lat")
    lon = el.get("lon") or el.get("center", {}).get("lon")
    if not lat: continue
    tags = el.get("tags", {})
    features.append({
        "type": "Feature",
        "geometry": {"type": "Point", "coordinates": [lon, lat]},
        "properties": {
            "name": tags.get("name", "Inconnu"),
            "operator": tags.get("operator", ""),
            "country": tags.get("addr:country", ""),
            "osm_id": el["id"]
        }
    })

geojson = {"type": "FeatureCollection", "features": features}
with open("data/datacenters_raw.geojson", "w") as f:
    json.dump(geojson, f, ensure_ascii=False, indent=2)

print(f"Extrait {len(features)} datacenters")
```

7.2 Initialisation Mapbox (main.js)

```
mapboxgl.accessToken = 'VOTRE_TOKEN_ICI';

const map = new mapboxgl.Map({
  container: 'map',
  style: 'mapbox://styles/mapbox/dark-v11',
  center: [0, 30],
  zoom: 2
});
```

```
map.on('load', async () => {
  const resp = await fetch('./data/datacenters.geojson');
  const data = await resp.json();

  map.addSource('datacenters', {
    type: 'geojson', data,
    cluster: true, clusterMaxZoom: 6, clusterRadius: 50
  });

  // Couche points individuels colorés par intensité carbone
  map.addLayer({
    id: 'dc-points', type: 'circle', source: 'datacenters',
    filter: ['!', ['has', 'point_count']],
    paint: {
      'circle-radius': ['interpolate', ['linear'],
        ['get', 'capacity_mw'], 0, 4, 500, 18],
      'circle-color': ['interpolate', ['linear'],
        ['get', 'carbon_intensity_gco2_kwh'],
        0, '#00C851',   // vert = renouvelable
        200, '#FFB300', // orange = mixte
        600, '#FF3547'  // rouge = fossile
      ],
      'circle-opacity': 0.85
    }
  });

  // Popup au clic
  map.on('click', 'dc-points', (e) => {
    const props = e.features[0].properties;
    new mapboxgl.Popup()
      .setLngLat(e.lngLat)
      .setHTML(buildPopupHTML(props))
      .addTo(map);
  });
});
```

8. Présentation jury — script 5 minutes

Timing	Moment	Contenu
0:00 – 0:45	🎯 **Accroche**	*« Vous avez utilisé ChatGPT ce matin. Quelque part sur Terre, un datacenter a consommé de l'électricité pour ça. Mais où exactement ? Et combien ? Personne ne vous le dit. »*
0:45 – 1:15	❓ **Problème**	Montrer le rapport Google : +48% d'émissions malgré les promesses. L'opacité est systémique.
1:15 – 3:30	🗺️ **Démo live**	Ouvrir la carte. Zoomer sur l'Europe. Cliquer sur un datacenter Google en Irlande (énergie verte) vs un en Virginie (charbon). Activer l'overlay stress hydrique. Montrer les stats globales.
3:30 – 4:15	💡 **Insight clé**	*« Ce datacenter consomme autant d'eau qu'une ville de 50 000 habitants. Il est implanté dans une zone en stress hydrique élevé. »*
4:15 – 5:00	🚀 **Impact et ouverture**	Données 100% open source. Outil citoyen gratuit. Perspective : intégration dans les outils de journalisme de données, éducation, politiques publiques.

Questions jury anticipées — réponses préparées
« Vos données sont fiables ? » → Ce sont des ordres de grandeur sourcés (OSM, rapports ESG officiels, IEA). L'opacité du secteur est elle-même un message : si les données étaient disponibles, notre outil serait plus précis.
« Pourquoi pas une extension comme prévu ? » → La carte permet de raconter une histoire géographique que l'extension ne peut pas. L'impact est territorialisé — c'est plus fort visuellement et analytiquement.
« Que manque-t-il ? » → Des données standardisées et obligatoires de la part des opérateurs. C'est précisément ce que des initiatives comme l'ADEME devraient pousser au niveau réglementaire.

9. Points de vigilance

⚠️ Risques techniques	✅ Bonnes pratiques
API Electricity Maps : limite de 50 req/h en gratuit → précharger les données carbone par pays au démarrage	Comprendre et valider le GeoJSON
OSM : qualité variable des tags 'operator' → prévoir nettoyage manuel pour les 50 grands	Versionner les données sous /data/ dans Git (pas de base de do
Token Mapbox : ne pas commiter dans Git → variable d'environnement ou fichier .env exclu	Séparer clairement les scripts de préparation (Python) du rendu
CORS sur GitHub Pages → les appels API doivent être depuis le client (pas de backend)	Tester la démo sur un réseau 4G simulé (jury en salle = WiFi par

9.1 Formule de calcul CO_2 — à documenter

Pour chaque datacenter, l'empreinte carbone annuelle est estimée selon :

$$CO_2 \text{ (tonnes/an)} = \text{Capacité (MW)} \times \text{PUE} \times 8760 \text{ h} \times \text{Intensité carbone } (gCO_2/kWh) \div 10^6$$

Exemple : 100 MW × 1.4 × 8760 h × 400 gCO_2/kWh ÷ 10^6 = 490 000 tonnes CO_2/an

9.2 Valeurs par défaut si données manquantes

Paramètre	Valeur par défaut	Source
PUE (non renseigné)	1.58 (moyenne mondiale IEA)	IEA Energy and AI 2025
Capacité MW (inconnue)	20 MW (médiane petits DCs)	Estimation équipe
Intensité carbone	API Electricity Maps par pays	electricitymaps.com
Consommation eau	1.8 L/kWh (ratio moyen)	Rapport Shift Project 2025

🌍 **DataCenter Impact Map**

Document de réalisation — Hack for Good × Défi ADEME

Données 100% open source · Outil citoyen · Pour le débat public